Master Prompt: Build a Full MERN Ride-Hailing App (Ola/Uber Clone)

Use this prompt as the **single source of truth** to design and implement a production-style ride-hailing web application using the MERN stack. The application must support **Passenger, Driver, and Admin** roles, JWT-based authentication, live ride tracking, fare estimation, ride sharing, driver verification, rating and review workflows, driver earnings, and machine-learning-based demand heatmap and dynamic pricing.[cite:1] [cite:3] [cite:6]

Role

Act as a senior full-stack architect and implementation agent. Build the complete application in a structured, scalable, and maintainable way using **MongoDB, Express.js, React, and Node.js**, with role-based access control and real-time communication for live tracking and ride lifecycle events.[cite:1] [cite:4] [cite:6]

Primary objective

Create a clean, modular, end-to-end MERN platform similar to Ola/Uber with these major capabilities:[cite:1] [cite:3] [cite:6]

- JWT signup and login for **Admin, Driver, and Passenger** with secure role-based authorization.[cite:2] [cite:5]
- Real-time ride booking and tracking with nearby driver matching, ETA updates, and ride state transitions.[cite:3] [cite:6]
- Fare estimation before booking and final fare calculation after trip completion.[cite:3] [cite:6]
- Ride sharing option with clear matching rules and seat availability logic.[cite:3]
- Driver background verification workflow with admin approval controls.[cite:3]
- Driver and passenger rating system after ride completion.[cite:3]
- Driver earnings dashboard with trip summaries, commission, payouts, and daily/weekly/monthly totals.[cite:3]
- Demand heatmap and dynamic pricing powered first by rule-based logic and then by machine-learning-ready data pipelines.[cite:3] [cite:6]

Non-negotiable rules

Follow these rules throughout execution:[cite:1] [cite:4] [cite:6]

1. Use a **clean architecture** approach with separation of concerns such as route, controller, service, model, middleware, and utility layers.[cite:1] [cite:4]

2. Keep frontend and backend modular so each role has isolated responsibilities and views. [cite:1] [cite:4]
3. Design for real-time communication using socket-based architecture for trip status and location updates.[cite:3] [cite:6]
4. Use role-based route protection both on the API side and UI side.[cite:2] [cite:5]
5. Make the project implementation-ready, but keep the structure readable and beginner-friendly.[cite:1]
6. Prefer scalable naming, consistent folder structure, and reusable services over shortcuts. [cite:1] [cite:4]
7. Start with a solid MVP and then layer advanced modules in phases.[cite:3] [cite:6]
8. Keep all business logic explicit; do not leave critical flows undefined or vague.[cite:1] [cite:6]

Product overview

The platform has three user types:[cite:2] [cite:3]

Role	Purpose
Passenger	Search rides, estimate fare, book rides, track trips, share rides, pay, and rate drivers. [cite:3]
Driver	Register, upload verification documents, go online, accept rides, navigate trips, view earnings, and receive ratings. [cite:3]
Admin	Verify drivers, manage users, monitor rides, resolve issues, view heatmaps, and control pricing intelligence. [cite:3][cite:6]

Required system modules

Build the app as the following modules.[cite:1] [cite:3] [cite:6]

1. Authentication and authorization

Implement secure signup and login for all three roles using JWT and role-based access control. Support password hashing, protected routes, middleware-based authorization, and role-aware dashboard routing.[cite:2] [cite:5]

Required flows:

- Passenger signup and login.[cite:2]
- Driver signup and login.[cite:2]
- Admin login and protected admin-only area.[cite:2] [cite:5]
- Access control middleware for Passenger, Driver, Admin.[cite:2] [cite:5]
- Account status checks, such as driver approved or blocked.[cite:3]

2. User and profile management

Create separate profile handling for Passenger, Driver, and Admin while preserving a unified authentication foundation. Store shared user identity in a base user record and keep role-specific details in dedicated role collections or modules for maintainability.[cite:1] [cite:5]

Required profile data:

- User identity: name, phone, email, password hash, role, status.[cite:2] [cite:5]
- Passenger profile: saved places, trip history, preferences.[cite:3]
- Driver profile: license, vehicle details, availability, verification state, current location, rating summary.[cite:3]
- Admin profile: system permissions and audit activity.[cite:2]

3. Ride booking engine

Build the full ride lifecycle from booking to completion. Include pickup, drop, fare estimate, driver search, ride request, ride acceptance, trip start, trip completion, cancellation, and trip history.[cite:3] [cite:6]

Required ride statuses:

- Requested.[cite:3]
- Searching driver.[cite:3]
- Driver assigned.[cite:6]
- Driver arrived.[cite:3]
- Trip started.[cite:3]
- Trip completed.[cite:3]
- Cancelled by passenger or driver.[cite:3]

Required ride features:

- Pickup and drop address capture.[cite:3]
- Nearby driver matching.[cite:6]
- ETA calculation and display.[cite:3] [cite:6]
- Fare estimate before confirmation.[cite:3] [cite:6]
- Final fare computation after completion.[cite:3]
- Trip history for both Passenger and Driver.[cite:3]

4. Real-time tracking

Implement a real-time system for live driver movement, ride progress updates, and state synchronization between Passenger, Driver, and Admin dashboards. The architecture must support frequent driver location updates and low-latency event broadcasting.[cite:3] [cite:6] [cite:9]

Real-time events must include:

- Driver online or offline update.[cite:3]
- Driver location update.[cite:6] [cite:9]
- Ride accepted or rejected.[cite:3]
- Driver arrived at pickup.[cite:3]
- Ride started.[cite:3]
- Ride completed.[cite:3]
- Ride cancelled.[cite:3]
- ETA refresh.[cite:3] [cite:6]

5. Ride sharing module

Support a ride-sharing option where compatible passengers can opt into shared rides. Keep this rule-driven and explicit. Matching should consider route similarity, seat availability, timing tolerance, and pickup-drop overlap.[cite:3]

Required ride-sharing controls:

- Shared ride toggle during booking.[cite:3]
- Vehicle seat capacity support.[cite:3]
- Ride grouping constraints and validation.[cite:3]
- Shared fare split logic.[cite:3]
- Shared ride visibility in Passenger and Driver trip views.[cite:3]

6. Driver verification workflow

Build a driver onboarding and approval system. A driver must not be allowed to go online until verification is completed and approved by Admin.[cite:3]

Required driver verification flow:

- Driver registration.[cite:2]
- Upload license and ID proof.[cite:3]
- Upload vehicle-related documents such as registration and insurance.[cite:3]
- Store verification status: pending, under review, approved, rejected.[cite:3]
- Admin review screen with approve or reject actions.[cite:3]

- Re-submission support after rejection.[cite:3]

7. Ratings and reviews

Enable both trust and quality scoring through post-trip reviews. At minimum, the passenger should rate the driver after ride completion, and optionally the driver may rate the passenger too.[cite:3]

Required rating features:

- Numerical rating per trip.[cite:3]
- Optional written feedback.[cite:3]
- Aggregated driver rating summary.[cite:3]
- Ratings shown in driver profile and ride assignment context if needed.[cite:3]

8. Driver earnings dashboard

Provide a driver dashboard that clearly shows financial performance. Include trip earnings, commissions, and payout status with simple, clear summaries.[cite:3]

Required earnings views:

- Today earnings.[cite:3]
- Weekly earnings.[cite:3]
- Monthly earnings.[cite:3]
- Total completed rides.[cite:3]
- Commission deducted.[cite:3]
- Pending and completed payouts.[cite:3]

9. Demand heatmap and dynamic pricing

Implement a pricing intelligence module in two layers. Start with basic rule-based surge logic, then prepare the system for ML-based demand prediction using trip history, time, area, supply, and request volume.[cite:3] [cite:6]

Required outputs:

- Area-wise demand heatmap for Admin.[cite:6]
- Surge multiplier suggestion based on demand and supply ratio.[cite:6]
- Historical ride volume analysis by zone and time slot.[cite:6]
- Machine-learning-ready dataset for future prediction.[cite:6]

Recommended architecture

Use a layered backend and role-oriented frontend. Keep the system organized as follows:[cite:1][cite:4][cite:6]

```
System Architecture
├── Frontend (React)
│   ├── Passenger App
│   ├── Driver App
│   └── Admin Panel
├── Backend (Node + Express)
│   ├── Auth Module
│   ├── User Module
│   ├── Ride Module
│   ├── Pricing Module
│   ├── Verification Module
│   ├── Ratings Module
│   ├── Earnings Module
│   └── Real-time Socket Module
├── Database (MongoDB)
│   ├── Users
│   ├── Drivers
│   ├── Passengers
│   ├── Rides
│   ├── Ratings
│   ├── Earnings
│   ├── VerificationDocs
│   ├── LocationLogs
│   └── DemandZones
└── ML/Analytics Layer
    ├── Heatmap generation
    ├── Surge logic
    └── Demand prediction pipeline
```

Database blueprint

Use MongoDB collections or models that reflect both operational and analytical needs. Include time-based and geo-based storage because nearby-driver lookup, heatmaps, and trip analysis rely on location and timestamped events.[cite:1][cite:3][cite:6]

Core collections

Collection	Key fields
Users	name, email, phone, passwordHash, role, status, createdAt. [cite:2][cite:5]
Passengers	userId, savedLocations, tripHistory, preferences. [cite:3]
Drivers	userId, vehicleInfo, licenseInfo, verificationStatus, availabilityStatus, currentLocation, averageRating. [cite:3]
Admins	userId, permissions, auditLogs. [cite:2]

Collection	Key fields
Rides	passengerId, driverId, rideType, pickup, drop, estimatedFare, finalFare, status, timestamps. [cite:3][cite:6]
Ratings	rideId, passengerId, driverId, score, reviewText, createdAt. [cite:3]
Earnings	driverId, rideId, grossAmount, commission, netAmount, payoutStatus, date. [cite:3]
VerificationDocs	driverId, documentType, fileRef, reviewStatus, remarks. [cite:3]
LocationLogs	driverId, rideId, latitude, longitude, speed, heading, timestamp. [cite:6][cite:9]
DemandZones	zoneId, geohashOrGrid, timeSlot, requestCount, activeDrivers, demandScore, surgeMultiplier. [cite:6]

Database design expectations

- Keep user identity separate from role-specific details.[cite:1][cite:5]
- Support geospatial querying for nearby driver discovery.[cite:6]
- Preserve ride status history for analytics and auditability.[cite:6]
- Store enough historical data to support future machine learning.[cite:6]

Backend execution plan

Build the backend in a modular and scalable manner. Suggested implementation order:[cite:1][cite:4][cite:6]

1. Project setup and environment configuration.[cite:1]
2. Database connection and base model design.[cite:1]
3. Auth module with JWT and role middleware.[cite:2][cite:5]
4. User and profile management module.[cite:1]
5. Ride booking engine.[cite:3][cite:6]
6. Real-time socket integration.[cite:3][cite:9]
7. Driver verification workflow.[cite:3]
8. Ratings and reviews.[cite:3]
9. Earnings aggregation logic.[cite:3]
10. Demand heatmap and pricing intelligence.[cite:6]

Backend quality requirements

- Use controllers only for request and response handling.[cite:1][cite:4]
- Put business logic inside services.[cite:1][cite:4]
- Keep validation centralized and reusable.[cite:1]
- Use middleware for auth, role checks, errors, and request validation.[cite:1][cite:5]
- Keep route files concise and feature-based.[cite:1]

- Ensure ride status transitions are validated and cannot skip illegally.[cite:3] [cite:6]

Frontend execution plan

Build the frontend as a role-based React application with clear dashboards and screen flows. The UI should stay clean, simple, and state-aware, with separate navigation for Passenger, Driver, and Admin.[cite:4]

Passenger screens

```
Passenger App
├── Auth
│   ├── Login
│   └── Signup
├── Home
│   ├── Pickup input
│   ├── Drop input
│   ├── Fare estimate
│   ├── Ride type selection
│   └── Share ride toggle
├── Driver Match
│   ├── Nearby drivers
│   ├── ETA
│   └── Confirm ride
├── Live Tracking
│   ├── Driver card
│   ├── Real-time map
│   ├── Ride status
│   └── Cancel / Support
├── Payment & Rating
└── Ride History
```

Driver screens

```
Driver App
├── Auth
│   ├── Login
│   └── Signup
├── Onboarding
│   ├── Personal details
│   ├── Vehicle details
│   ├── Document upload
│   └── Verification status
├── Driver Dashboard
│   ├── Go online/offline
│   ├── Incoming requests
│   ├── Active trip panel
│   └── Navigation / status controls
├── Earnings Dashboard
│   ├── Daily
│   ├── Weekly
│   ├── Monthly
```

```
│   └── Payout status
└── Ratings & Profile
```

Admin screens

```
Admin Panel
├── Admin Login
├── Dashboard
│   ├── Total users
│   ├── Active drivers
│   ├── Ongoing rides
│   └── Revenue summary
├── Driver Verification Queue
├── User Management
├── Ride Monitoring
├── Ratings / Complaints
├── Demand Heatmap
└── Dynamic Pricing Control
```

API design expectations

Create a clean REST API structure with feature-based grouping. Ensure the API covers auth, users, rides, verification, ratings, earnings, and analytics in a predictable way.[cite:1] [cite:4] [cite:6]

Suggested endpoint groups:

- `/auth`
- `/users`
- `/drivers`
- `/passengers`
- `/rides`
- `/ratings`
- `/earnings`
- `/verification`
- `/admin`
- `/pricing`
- `/analytics`

For each group, define:

- Route purpose.[cite:1]
- Request and response contracts.[cite:1]
- Authorization requirements.[cite:2] [cite:5]
- Validation rules.[cite:1]

- Success and error states.[cite:1]

Real-time logic expectations

The system must clearly model socket events and ownership of each event. Real-time logic should not be vague; define event names, emitters, listeners, and affected roles.[cite:3] [cite:6] [cite:9]

Required event map:

- Driver connects and sends availability.[cite:3]
- Driver emits periodic location updates.[cite:6] [cite:9]
- Passenger subscribes to assigned ride updates.[cite:3]
- Admin subscribes to active ride monitoring feed.[cite:3]
- Ride state changes broadcast to relevant users only.[cite:3] [cite:6]

Pricing and ML execution expectations

Implement pricing in stages so the app works before advanced prediction is added. Begin with deterministic logic and then introduce analytics and machine-learning-readiness.[cite:3] [cite:6]

Stage 1: Rule-based pricing

- Base fare.[cite:3]
- Distance fare.[cite:3]
- Time fare.[cite:3]
- Waiting charge.[cite:3]
- Surge multiplier using active demand versus active supply.[cite:6]

Stage 2: Demand heatmap

- Divide service area into zones or grids.[cite:6]
- Count ride requests by zone and time window.[cite:6]
- Show high-demand regions on admin view.[cite:6]

Stage 3: ML-ready demand prediction

- Collect features such as zone, hour, weekday, demand count, active drivers, completed rides, and cancellations.[cite:6]
- Produce demand score predictions per zone and time slot.[cite:6]
- Convert predicted demand into surge multiplier suggestions.[cite:6]

Security expectations

Security must be treated as part of the core build. JWT and role-based auth are only the starting point.[cite:2][cite:5]

Required security practices:

- Password hashing.[cite:2][cite:5]
- Access token validation.[cite:2][cite:5]
- Role-based authorization.[cite:2][cite:5]
- Protected admin routes.[cite:2][cite:5]
- Driver approval gate before accepting rides.[cite:3]
- Input validation and error handling.[cite:1]
- Secure handling of driver documents.[cite:3]

Implementation phases

Follow this exact build sequence to reduce risk and avoid broken dependencies.[cite:1][cite:3][cite:6]

Phase 1: Planning and setup

- Initialize frontend and backend projects.[cite:1]
- Finalize folder structure.[cite:1][cite:4]
- Define database schema and role matrix.[cite:1][cite:5]
- Prepare environment configuration.[cite:1]

Phase 2: Authentication foundation

- Implement signup and login.[cite:2][cite:5]
- Add JWT handling.[cite:2][cite:5]
- Add role-based route guards.[cite:2][cite:5]
- Create base dashboards per role.[cite:4]

Phase 3: Core ride flow

- Implement booking form and ride creation.[cite:3]
- Add fare estimate service.[cite:3][cite:6]
- Add driver discovery and assignment.[cite:6]
- Add ride lifecycle state handling.[cite:3]

Phase 4: Real-time layer

- Add sockets.[cite:3] [cite:9]
- Add live location broadcasting.[cite:6] [cite:9]
- Add live passenger tracking UI.[cite:3]
- Add admin ride monitoring.[cite:3]

Phase 5: Trust and financial modules

- Add driver verification flow.[cite:3]
- Add ratings and review flow.[cite:3]
- Add earnings dashboard and payout state.[cite:3]

Phase 6: Smart analytics

- Add demand heatmap.[cite:6]
- Add surge logic.[cite:6]
- Add machine-learning-ready data pipeline.[cite:6]

Required deliverables

The final output should include all of the following :[cite:1] [cite:4] [cite:6]

- Complete project folder structure.[cite:1]
- Frontend pages and role-based navigation plan.[cite:4]
- Backend module breakdown.[cite:1] [cite:4]
- Database schema design.[cite:1] [cite:6]
- API design map.[cite:1]
- Socket event plan.[cite:3] [cite:9]
- Ride flow state diagram or equivalent textual state machine.[cite:3] [cite:6]
- Driver verification workflow.[cite:3]
- Ratings and earnings workflow.[cite:3]
- Pricing and heatmap logic design.[cite:6]
- Step-by-step implementation order.[cite:1] [cite:6]

Output format required from the implementation AI

Return the work in this order:

1. High-level architecture.[cite:1] [cite:4]
2. Folder structure.[cite:1]
3. Database schema definitions.[cite:1] [cite:6]

4. Backend modules and route plan.[cite:1][cite:4]
5. Frontend page structure.[cite:4]
6. Wireframes in markdown tree format.[cite:4]
7. Ride lifecycle logic.[cite:3][cite:6]
8. Real-time event flow.[cite:3][cite:9]
9. Pricing and ML flow.[cite:6]
10. Phase-wise execution roadmap.[cite:1][cite:6]
11. Then begin implementation in a clean, modular, production-style manner.[cite:1][cite:4]

Final instruction to the implementation AI

Do not skip architecture. Do not jump directly into random code. First define the full system clearly, then implement it module by module with consistent naming, reusable patterns, protected routes, and explicit ride-state logic. Every critical area such as auth, matching, tracking, verification, ratings, earnings, and dynamic pricing must be complete, connected, and logically sound.[cite:1][cite:3][cite:6]